

LLM 工程 / 智能体 / 应用型 AI

打造面向真实业务的可靠 LLM 系统。

我是 SX Qian，专注于多智能体 workflow、检索系统、结构化生成、模型适配和评估驱动的 AI 产品。

Multi-Agent	RAG	PyTorch	LoRA	Evaluation	Deployment
-------------	-----	---------	------	------------	------------

01	企业级金融合规多智能体系统	脱敏企业案例
02	因果推断领域 RAG 系统	公开 GitHub 项目
03	Qwen 视觉模型 OCR 微调	公开 GitHub 项目

工程方法

我通过输出约束、中间验证、安全执行、自动化评估和证据驱动迭代，让模型行为可测试、可追踪。

企业级金融合规多智能体系统

面向真实业务的多智能体系统，将复杂政策文档转换为结构化规则、可执行验证逻辑、风险分析和合规报告。

核心挑战

难点并不只是让多个 Agent 相互调用，而是保证每个中间输出在政策表达模糊和数据映射不完整的情况下仍然结构化、可执行、可验证、可追踪。

系统流程

政策文本	规则理解	规则 Schema	数据匹配	验证器	风险分析	报告
------	------	-----------	------	-----	------	----

我的主要工作

规则结构化	定义统一规则字段和数据约束，将自然语言政策转换为一致的 Rule Schema。
可执行验证	将结构化规则编译为 Python validators，并使用 AST 检查和自动化测试验证生成逻辑。
AGENT 可靠性	引入受约束输出、Schema 校验、自动重试和错误恢复，并交叉验证执行结果与模型结论。
评估体系	从 Schema 有效性、字段映射、单元测试、风险识别、合规验证、标签一致性和虚构字段等维度评估系统。
推理服务	使用 vLLM 和 Docker 部署推理服务，并针对不同 Agent 阶段测试并发参数。

Python	Pydantic	JSON Schema	AST	vLLM	Docker
--------	----------	-------------	-----	------	--------

不只依赖 Prompt 的工程设计

早期只依赖 Prompt 控制格式时，系统容易出现字段缺失、JSON 不完整和结构不一致。后续方案将模型约束与确定性软件检查结合起来。

输出契约	通过 JSON Schema constrained decoding 和 Pydantic validation 建立可机器校验的 Agent 接口。
错误恢复	无效输出触发结构化重试和显式错误处理，避免错误静默传播到下游。
代码安全	生成的验证器先经过解析和 AST 检查，再通过单元测试验证。
交叉验证	对比 Python 执行结果与模型结论，发现缺少依据的合规判断。

数据集与模型优化

参与构建政策文本、结构化规则、验证逻辑、单元测试和风险/合规标签等训练与评估数据，并探索 SFT、LoRA、DPO 和 RFT，以改善规则解析、结构化生成、验证代码正确性和标签一致性。

并发实验与取舍

并发实验提升了整体吞吐量，同时也暴露了少量结果漂移，包括在缺少必要关联键时出现错误合规判断。因此系统没有使用统一并发值，而是针对不同 Agent 阶段分别设置限制，在速度和稳定性之间取得平衡。

核心认识

企业级 Agent 系统的可靠性来自明确的输出契约、验证机制、可观察的失败路径，以及超越最终文本相似度的评估体系。

项目 02 / 公开 GITHUB

因果推断领域 RAG 系统

[打开 GitHub 仓库 >](#)

面向因果推断知识和推荐分析问题的检索增强助手，在生成答案前先从领域文档中进行本地向量检索。

项目问题

通用语言模型可以给出流畅解释，但答案未必基于项目资料。该工作流通过检索层引入相关领域上下文，使回答获得更明确的知识依据。

处理流程

领域文档	文本切分	Embedding	FAISS	Top-k 上下文	Gemini	回答
知识来源	准备因果推断和推荐分析相关的领域材料，作为检索知识库。					
向量表示	使用 Sentence Transformers 将文本块和用户问题转换为向量。					
向量索引	通过 FAISS 实现本地相似度检索，无需依赖托管向量数据库。					
答案生成	将检索上下文组合到 Prompt 中，再交给 Gemini 生成有依据的回答。					
流程编排	使用 LangChain 连接文档处理、检索、Prompt 和答案生成环节。					
Python	LangChain	Sentence Transformers	FAISS	Gemini		

项目 02 / 设计与评估

这个 RAG 项目体现了什么

该项目展示了如何围绕具体分析领域改造通用 RAG 流程，而不是把它简单当作一个万能聊天机器人。

领域适配	检索内容和 Prompt 围绕因果推断术语及推荐分析使用场景组织。
本地检索	FAISS 使向量索引保持轻量，便于本地实验和复现。
模块解耦	Embedding、索引、检索和生成相互分离，便于独立检查和替换。
有依据生成	生成模型使用检索上下文，而不是只依赖模型参数中的记忆。

评估思路

完善的 RAG 评估应区分检索质量和生成质量。检索环节可检查返回文本块的相关性、覆盖度和多样性；答案环节可检查依据性、完整性、领域正确性和无依据陈述。

检索检查	返回文本块是否相关、足够且避免重复？
答案检查	回答是否正确使用上下文，并避免超出证据的结论？
失败分析	面对歧义问题、领域外问题或无检索结果时，系统如何表现？
后续扩展	元数据过滤、reranking、引用展示、query rewriting 和固定评估集。

岗位相关性	该项目体现了实际 RAG 架构、本地向量检索、领域适配，以及区分检索质量与答案质量的评估意识。
-------	---

在 GitHub 查看源码、README 和项目文件 >

项目 03 / 公开 GITHUB

Qwen3.5 视觉模型 LaTeX OCR 微调

[打开 GitHub 仓库 >](#)

使用 LoRA 将 Qwen3.5 系列视觉语言模型适配至图像到 LaTeX 生成任务的端到端 PyTorch 工作流。

项目目标

完整理解多模态微调流程，包括数据准备、视觉文本对齐、参数高效适配、分布式训练、断点恢复、指标记录和注意力分析，而不只是运行一次推理。

训练配置

基础模型	Qwen3.5-VL 4B，使用 LoRA 进行参数高效适配。
数据集	unsloth/LaTeX_OCR，包含 68,686 组图像与 LaTeX 文本。
LORA 参数	Rank 8、alpha 16、dropout 0.05。
计算环境	使用 PyTorch Distributed Data Parallel，在两张 NVIDIA V100 16 GB GPU 上训练。
显存策略	使用 gradient checkpointing、关闭 use_cache、小批量训练和分块数据处理。
训练恢复	通过可恢复 checkpoint 支持跨多个集群会话继续训练。

PyTorch	Qwen3.5-VL	LoRA	DDP	多模态	LaTeX OCR
---------	------------	------	-----	-----	-----------

项目 03 / 数据、训练与分析

构建可复现的多模态训练流程

仓库将环境配置、数据、模型、训练、分布式训练、评估和注意力分析拆分为可复用的 Notebook 与 Python 模块。

输入构造	图像经过缩放并转换为 vision tokens，再与 prompt tokens 和目标 LaTeX 序列对齐。
序列规划	通过 token 长度统计确定 384-token 限制，减少不必要的截断。
分块训练	将数据集分块处理，使大规模训练适应集群显存和会话时长限制。
实验记录	以 JSON、CSV 和 PNG 格式导出指标与实验产物。
模型评估	在未见样本上评估模型质量，并通过 attention visualization 分析模型行为。
训练表现	公开实验记录显示损失在早期下降后趋于稳定，未出现 NaN loss 或梯度爆炸。

仓库流程

环境配置	数据	模型	训练	DDP	评估	注意力分析
------	----	----	----	-----	----	-------

能力体现	该项目体现了多模态模型适配、GPU 资源约束下的训练设计、可恢复实验、PyTorch 分布式训练，以及超越单个输出样例的评估意识。
------	---

工程思考

项目将序列长度、图像尺寸、batch size、checkpoint、chunk size 和跨会话恢复都作为明确的设计参数。这些因素决定了模型微调能否从 Notebook 演示转化为可重复运行的训练 workflow。

在 [GitHub](#) 查看 Notebook、训练代码和 README >

邮箱	sxqian@bu.edu
GITHUB	github.com/sxqian-lgtm